

Apex Terminal Two-Year Master Plan (Week-by-Week)

Current Status & Assumptions: *After an initial burst of ~5–7 days covering 18 rapid “waves” of development, many foundational elements are already in place.* UI2 is now the default interface, a deterministic **Demo mode** runs end-to-end, and initial **Autopilot**/automation back-end wiring is live. **Telemetry** and **search** features are functioning, and each wave has been delivered with rigorous testing (unit tests, integration tests via Vitest/Pytest, end-to-end tests with Playwright) and evidence packs. This accelerated progress puts the project well ahead of a typical schedule. **Going forward, this 2-year master plan assumes the team maintains that momentum**, but with a focus on deeper automation, thorough testing at every step, and more comprehensive improvements. The plan is organized by quarter, with each week's objectives listed, ensuring every feature is implemented and validated thoroughly.

Year 1 – Q1: Stabilization & Deployment Foundations (Weeks 1–13)

- **Week 1:** Implement a canonical *symbol model* (e.g. a **SymbolRef** data structure) and a symbol resolver API for consistent instrument identification; add corresponding unit tests and end-to-end tests; integrate the resolver with the new UI2 interface.
- **Week 2:** Design and implement a **Watchlist** panel with full CRUD operations (create, edit, delete watchlists); build watchlist management APIs; integrate watchlist functionality into UI2 so users can view and modify their watchlists.
- **Week 3:** Extend watchlist functionality to support *multiple watchlists* per user and add quick actions (e.g. one-click to open analytics or run backtests on watchlist items); ensure proper validation by creating automated UI tests (using Playwright) for multi-watchlist workflows.
- **Week 4:** Develop a deterministic **Alerts** engine capable of monitoring predefined conditions (price thresholds, volume spikes, etc.); create an **Alerts Board** in UI2 that displays triggered alerts with filters (by symbol, type, time); include unit tests for the alerts logic to ensure no false triggers.
- **Week 5:** Link the new Alerts system with watchlists – e.g. allow users to set alerts on watchlist items. Enable alert *triggers* that perform actions when conditions are met (e.g. send notification or execute trade). Add necessary API endpoints for creating and managing alerts, and include tests to verify alert triggers and notifications.
- **Week 6:** Refine the **symbol resolver UI** in the platform – enforce a disambiguation modal for symbols with multiple matches (e.g. same ticker in different exchanges); add a search-as-you-type autocomplete feature in symbol search. Ensure the UI changes are thoroughly tested for accessibility and correctness.
- **Week 7:** Create a centralized **Event Log** service to record all system events (orders executed, trades filled, alerts triggered, errors/incidents, etc.) with deterministic timestamps and fields. This service will form the basis for audit trails.
- **Week 8:** Build an **Event Log** panel in UI2 that allows users and admins to browse the event history; include search and filter capabilities (by event type, severity, time range) for easy navigation of logs. Write integration tests to confirm that events are logged and displayed correctly in the UI.

- **Week 9:** Add a time-series API to enable *event replay* – the ability to retrieve sequences of past events over time. Design playback controls in UI2 (e.g. a timeline scrubber) to replay events for a given period, helping with debugging or analysis of past system behavior.
- **Week 10:** Develop **scenario-driven alert templates** for common scenarios (e.g. a template for price breakouts, volume surges, etc.) that users can easily configure. Integrate these alert templates into the watchlist and alerts UI, so that adding a watchlist item can suggest relevant alert rules.
- **Week 11:** Implement a *multi-workspace* feature allowing users to have multiple dashboard layouts (workspaces) saved. Introduce persistent layout storage so that a user's workspace arrangement (panel positions, sizes, etc.) is saved between sessions. Create a **Workspace Manager** API to handle saving, loading, and listing available workspaces.
- **Week 12:** Add UI2 controls for workspace management – allow users to **save**, **load**, or **reset** their dashboards to default. Ensure that switching workspaces is smooth and state (like open panels, selected symbols) is preserved. Test these features for deterministic behavior (e.g. the same layout is restored every time for a given workspace).
- **Week 13:** Introduce profile-based *layout defaults* – e.g. provide preset workspaces for different purposes (a "Trading" layout, a "Research" layout, etc.) that new users can start with. Implement the logic to load these default layouts based on user role or preference. Add end-to-end tests to verify that defaults load correctly and can be customized by the user.

Year 1 – Q2: Research/Analytics Depth & Data Quality (Weeks 14–26)

- **Week 14:** Redesign the application's left-side navigation (**LeftNav**) to accommodate dynamic workspace groups and new features. This may include a hierarchical menu or tabs to switch between different workspaces or major sections. Ensure the redesigned nav is accessible (keyboard navigation, screen-reader labels) and intuitive.
- **Week 15:** Introduce **personalized dashboards** – allow users to pin their preferred components (e.g. a particular watchlist, an orders table, or a specific chart) to a personal dashboard. This involves adding a pinning feature on panels and a screen to arrange pinned items.
- **Week 16:** Expand the search functionality to index additional objects/artifacts in the system, such as saved strategies, backtest results, and run artifacts. Refactor the search storage/index to handle these new object types efficiently. This ensures that the global search bar can return results not just for symbols, but also for user-created content (strategies, past analysis runs, etc.).
- **Week 17:** Integrate search results with UI navigation – clicking a search result should navigate the user to the relevant panel or context (e.g. opening the strategy editor if a strategy was found, or focusing the chart if a symbol was found). This requires mapping result types to UI actions. Test end-to-end that each result type correctly deep-links into the app.
- **Week 18:** Add **full-text search** capability on telemetry and log events. This means users (or devs) can search within the contents of log messages, error descriptions, etc. Implement robust filtering options (by event type, severity, time window) to narrow search results. Ensure that indexing and querying are efficient given potentially large log volumes.
- **Week 19:** Implement *saved search* functionality and search history. Users should be able to save frequent searches (including filters) and quickly re-run them. Provide a UI component to manage saved searches (rename, delete) and show recent search history for convenience.
- **Week 20:** Improve search relevance via ranking enhancements. Implement sorting of search results by relevance, using factors like frequency or recency. Add support for synonyms and stop-word filtering in search queries (e.g. treat "USA" and "United States" as equivalent in search, ignore common words like "the").

- **Week 21:** Optimize the UI tables that may contain very large datasets (e.g. telemetry logs, orders, trades). Integrate **virtual scrolling** or windowing so that only visible rows are rendered, ensuring smooth performance with thousands of rows. Test this with large dummy data to verify that performance and memory usage remain within acceptable bounds.
- **Week 22:** Implement pagination in backend API endpoints that return large lists (orders, trades, events, etc.) to complement front-end virtualization. Ensure each paginated response has deterministic ordering (e.g. sorted by timestamp or ID) so that data doesn't jump around. Create tests using large fixtures to verify pagination works correctly and edge cases (start/end of data) are handled.
- **Week 23:** Optimize charting performance for long time series. Implement downsampling or aggregation for chart data when zoomed out so that rendering remains fast even for very large datasets. Fix any issues like axis flicker or high CPU usage when many data points are present.
- **Week 24:** Improve data retrieval for **backtests** and **risk simulations** by implementing chunked or streaming data loading. This prevents UI freezing when loading very large result sets. Integrate this with UI2 so that backtest results or risk analysis outcomes load incrementally (with a spinner/progress if needed).
- **Week 25:** Refactor the application's caching layer. Unify local storage and in-memory caching across modules (trading data, telemetry logs, analytics results) so that there's a single source of truth and less duplicate fetching. For example, if an asset price history is cached for charts, the backtester can reuse it. This improves consistency and reduces load.
- **Week 26:** Implement a **Compliance & Audit bundle (V1)** – a packaged export that includes all key data for compliance (order histories, position snapshots, logs, autopilot decisions, etc.). Essentially, create a mechanism to generate an audit report or data dump covering a period of activity. Include all necessary data in a signed, tamper-evident format for external audit or regulatory review.

Year 1 – Q3: Automation & Autopilot Maturity (Weeks 27–39)

- **Week 27:** Integrate the audit bundle export (from Week 26) into the UI's **Export** panel. Allow an authorized user to select a date range and generate the compliance bundle, then download it. Provide an offline verification tool or viewer to open the bundle and verify signatures/checksums to ensure data integrity.
- **Week 28:** Add **role-based access control (RBAC)** in the backend. Define roles (e.g. Admin, Trader, Viewer) and permissions for each major feature (trading, configuration, admin functions). Implement user authentication tokens that carry role information. Ensure all API endpoints enforce role permissions (for example, only Admin can export audit bundles or manage users).
- **Week 29:** Implement a user login/logout and session management flow in UI2. Connect to the RBAC backend from Week 28: upon login, retrieve the user's role and adjust UI accordingly (hide/disable ineligible sections). Ensure that certain panels or actions (like user management, audit exports) are visible only to authorized roles.
- **Week 30:** Develop an **Audit Log** UI component. This is distinct from the event log; it specifically records user actions (logins, configuration changes, critical operations). In UI2, provide an interface for admins to review the audit log, with sorting and searching by user or action type.
- **Week 31:** Create a **plugin interface** to allow custom modules (particularly for risk models or decision logic) to be added to the system. This involves defining a clear API for plugins and points in the system where they can hook in (e.g. risk evaluation step, or autopilot decision override). Expose hooks that the **Autopilot** can call, so that advanced users or internal developers can inject custom behavior.

- **Week 32:** Develop a **Plugin Manager** UI in UI2. This will list available approved plugins, allow installation or removal of plugins, and show their status. Only admin users can access this. The Plugin Manager communicates with the backend to load/unload plugin modules safely (possibly requiring a restart or sandboxing).
- **Week 33:** Publish a sample plugin (for example, a custom risk-check module) to demonstrate the plugin system. Provide documentation and a testing sandbox for plugin developers. Internally, write comprehensive tests for the plugin API to ensure that faulty plugins cannot break system determinism or stability.
- **Week 34:** Integrate plugin outputs into the telemetry/logging system. For instance, if a plugin performs risk checks or decisions, its actions and results should be logged just like native system actions. Ensure that plugin-generated events are clearly marked and that their output is deterministic (so that running the same input through a plugin yields the same logged result, aiding in debugging).
- **Week 35:** Conduct a UI/UX **consistency pass** on UI2. Unify styling (spacing, font sizes, color palette) across all panels and components to ensure a polished, professional look. Update style variables or tokens to enforce consistency. Fix any usability issues identified in earlier features (such as misaligned buttons or unclear labels).
- **Week 36:** Extend **Autopilot** for multi-user operation in a multi-portfolio environment. Ensure that Autopilot can manage separate portfolios for different users without overlap (portfolio segregation). Implement a *user-level “kill switch”* so that any user can pause their Autopilot if needed (and admins can globally pause all). Test Autopilot behavior with multiple concurrent users to verify isolation and thread safety.
- **Week 37:** Implement a user invitation and approval workflow to support adding new users to the platform (especially if this will be used in a team or enterprise context). For example, an admin can invite a user by email, the user signs up, and the admin approves and assigns a role. Add tests to cover the various flows (invite, accept, reject, role assignment).
- **Week 38:** **Polish compliance features** introduced so far. This includes ensuring the audit logs, compliance bundle, and other governance-related features are complete and user-friendly. Add the ability to export a summary of user actions or plugin usage for compliance. Write tests to verify that all compliance requirements (data retention, export integrity, access restrictions) are being met as expected.
- **Week 39:** Introduce **cross-currency trading support**. If the platform initially focused on a single currency (e.g. USD), extend it to support trading assets in multiple base currencies. This involves updating order entry, PnL calculations, and risk metrics to handle currency conversion. Add UI fields where necessary (to select currency, show FX rates) and include tests to verify correct currency conversion and rounding in trades and portfolios.

Year 1 – Q4: Enterprise Hardening & Extensibility (Weeks 40–52)

- **Week 40:** Implement *cross-asset exposure* calculations in the risk engine (e.g. if the platform now supports equities, options, crypto, etc., ensure the risk engine can aggregate exposures across asset classes). This may involve normalizing positions to a common risk unit or handling asset-specific risk parameters. Test that risk reports correctly reflect diverse holdings and that limits can be set per asset class or overall.
- **Week 41:** Integrate real-time streaming improvements. Upgrade the system to use **WebSockets** for live updates across all modules (prices, alerts, logs, etc.) rather than polling. Implement heartbeat

and reconnection logic for reliability. This ensures that data on the UI is updating instantaneously (e.g. live price ticks, real-time alerts).

- **Week 42:** Develop **advanced charting** features to meet enterprise-grade analytics needs. Introduce support for multi-asset overlays on a single chart (e.g. compare a stock's price vs. an index or vs. another stock), add a mini-map (small overview chart of the full data range), and annotation tools (for marking events or notes on charts).
- **Week 43:** Introduce new **data ingestion connectors** for external data sources. For example, integrate an alternative market data provider or a news feed API. Ensure the platform can handle multiple data providers simultaneously (perhaps one for equities, one for crypto, etc.). Create fixtures and tests for these connectors to simulate data and ensure the system can switch providers or use them in parallel.
- **Week 44:** Build a **portfolio backtester** that supports running multiple strategies or multiple assets simultaneously. Enable parameter sweeps (running a strategy with a range of parameters to see which performs best). This will likely be a substantial feature: design a job system for backtests if not already present, and ensure the UI can configure and display multi-strategy results (maybe comparative charts and tables).
- **Week 45:** Enable **scheduled backtests** – allow users to schedule backtest jobs to run at certain times or on a recurring basis. This requires a scheduling component (cron-like service or leveraging a cloud scheduler). Add UI elements for scheduling (pick frequency, next run time, etc.) and ensure the backtest runner can queue and execute jobs concurrently without interference. Write concurrency tests to verify multiple backtests can run in parallel and results remain correct.
- **Week 46:** Proactively address performance bottlenecks identified so far. This includes adding memoization or smarter caching for frequently computed results, optimizing WebSocket message handling (e.g. batch updates instead of sending too many small messages), and tuning database queries or indexes for faster responses. The goal is to keep the app responsive under heavy load.
- **Week 47:** Release a set of **compliance enhancements** based on any gaps found. For instance, implement digital signing of all data exports (reports, audit bundles) and add cryptographic checksums to important logs to detect tampering. This provides higher confidence for enterprise and regulatory use. Verify these features with tests that attempt to alter a signed file and ensure the system detects it.
- **Week 48: Public Release Preparation.** Tackle all final bug fixes and run a comprehensive end-to-end smoke test covering all major flows (user signup, trading, analytics, autopilot activation, etc.). Freeze any new feature development this week to focus on stability. By the end of Week 48, aim to have a release candidate build that could be delivered to external users or clients.
- **Week 49:** Expand **multi-provider data** capabilities. Allow the platform to connect to and use multiple market data providers simultaneously for redundancy or breadth of coverage. For example, if one provider lacks data for a particular asset, the system can pull from an alternate source. This may involve a layer to manage provider priority and merging of data feeds.
- **Week 50:** Build an **aggregator** for cross-exchange or cross-provider price feeds. If an asset trades on multiple exchanges or multiple sources provide quotes, aggregate them to present a unified order book or a unified price feed (best bid/ask across sources). Ensure time synchronization and handle duplicates.
- **Week 51:** Develop advanced **portfolio risk metrics** for enterprise users: e.g. Value-at-Risk (VaR), Conditional VaR, Beta to major indices, etc. Add these calculations to the risk engine and display the results in UI2 through new risk dashboards or charts. Provide context so that risk managers can understand the exposure (e.g. sector breakdown, correlation heatmaps if applicable).
- **Week 52: Enhance Autopilot** with macro-level signals. Integrate data for trending macro indices or economic indicators (like S&P 500 index levels, volatility index, interest rates) into Autopilot's

decision-making. Implement a *pre-market scanning* feature where Autopilot analyzes market conditions before the trading session (using those macro signals and any news/indicator feed) to adjust its strategy for the day. This concludes Year 1 with Autopilot becoming more context-aware.

Year 2 – Q1: Integrations, Marketplace & Multi-Tenancy (Weeks 53–65)

- **Week 53:** Begin integrating **predictive modeling** capabilities. Establish connectors or stub APIs to a forecasting models repository (for example, hooking up to a machine learning service that provides price or risk forecasts). This week is exploratory – set up the scaffolding that will allow plugging in predictive models into the system in later weeks.
- **Week 54:** Create an **optimization engine** for portfolio allocation, to work with Autopilot. Implement multi-objective optimization (e.g. maximizing return while minimizing risk or tracking error) for how Autopilot sizes positions and allocates assets. This likely involves linear or quadratic programming or using an optimization library. Ensure it's generic enough to plug in different objective functions.
- **Week 55:** Enhance the **risk scenario** engine. Add support for custom scenario construction – users or risk managers can define hypothetical scenarios (e.g. “what if interest rates rise 1% and equity markets drop 10%?”) and see the projected impact. Also enable chaining scenarios in sequence (multi-shock scenarios) to simulate prolonged stress events.
- **Week 56:** Develop **dynamic risk budgeting** for Autopilot strategies. This means Autopilot can adjust how much risk (or capital) to allocate to a strategy based on market volatility regimes or other indicators. For instance, in high volatility, it might automatically scale down position sizes. Implement the logic to monitor volatility (perhaps via an internal volatility index or ATR) and adjust strategy parameters on the fly.
- **Week 57:** Introduce a *multi-asset aggregator* for risk and performance. If the user's portfolio spans multiple asset classes and markets, create unified views that aggregate metrics across all assets. For example, allow a combined view of risk that includes equities, crypto, etc., and unify trade logs across markets. This ensures that the presence of multiple asset types doesn't silo the data – the user can still see a holistic picture.
- **Week 58:** Implement **margin requirements & leverage management** features. For margin-traded assets, calculate margin requirements for each position and overall (initial vs maintenance margin). Provide the UI and backend logic for users to see their margin utilization. Add the ability to set leverage limits and generate warnings or automatic de-leveraging if limits are breached.
- **Week 59:** Expand the plugin API (from Q3 Year1) to support third-party **analytics modules**. For example, allow plugins that add new analytics charts or data analyses in the UI. Update the Plugin Manager to handle these new plugin types and ensure security (e.g. plugins can't access data outside their scope). Provide updated documentation for developers on how to create analytic or UI plugins.
- **Week 60:** Add watchlist-based Autopilot triggers. This feature allows Autopilot to automatically propose or initiate strategies based on watchlist events – for instance, if a stock on the watchlist has a big earnings beat, Autopilot could suggest a trade. Implement the logic to monitor watchlist items for significant events and generate **Autopilot suggestions** or actions.
- **Week 61:** Create an **AI-driven insights dashboard**. This dashboard will compile key metrics and patterns across the user's portfolios and strategies, possibly generating natural-language summaries of performance or anomalies. Use a deterministic large language model (LLM) or rule-based system to ensure reproducible outputs (for compliance reasons). Integrate the insights with

the rest of the platform – for example, an insight might suggest enabling an Autopilot strategy or adjusting a risk setting, linking directly to those modules.

- **Week 62:** Develop enhanced **user alert customization** options. Let users define custom conditions for alerts with more granularity (for example, combining multiple conditions: “Alert me if stock X drops 5% AND volume doubles”). Allow these complex alerts to trigger various notification channels (email, SMS, in-app). This will likely involve a more advanced rule builder UI and corresponding logic on the backend.
- **Week 63:** Integrate **advanced chart analytics** tools. For example, add overlays for moving correlations between assets, generate implied volatility surface charts for options, or allow multi-dimensional charting (like surface or contour plots for strategies). These tools help power users and researchers perform in-depth analysis directly in the platform without exporting data to external programs.
- **Week 64:** Implement a **trade reconciliation** and P&L attribution tool. This would compare executed orders/trades against expected or theoretical outcomes. For instance, verify that all intended orders were executed (none missed or duplicated), highlight any discrepancies, and calculate P&L differences due to slippage or delays. Display reconciliation results in UI2, possibly in a dedicated report or dashboard.
- **Week 65:** Expand the **compliance report pack** introduced earlier to include periodic summary reports. Generate daily and monthly compliance summaries (e.g. end-of-day reports for trading activity, end-of-month aggregated risk and performance metrics) that can be reviewed and signed off by compliance officers. Implement a sign-off workflow where designated users can mark a report as reviewed/approved, with an audit trail.

Year 2 – Q2: Performance, Scale & Reliability (Weeks 66–78)

- **Week 66:** Enable cross-platform and responsive support for the UI. Adapt UI2 layouts for different screen sizes – ensure it works well on large monitors (e.g. multi-column layouts) and is responsive down to tablet dimensions. This may involve creating responsive CSS or adding alternate layouts for certain components.
- **Week 67:** Begin a **mobile-friendly** design initiative. While a full mobile app may be outside scope, ensure the core dashboards and status views have mobile-responsive versions or a pared-down UI that can at least display key info on a phone. Throughout these adaptations, maintain deterministic behavior in the application state and data flows (to avoid any non-reproducible issues introduced by asynchronous mobile optimizations).
- **Week 68:** Add a **public API** for third-party integration. Expose secure endpoints so that external applications or scripts can retrieve data (e.g. portfolio holdings, historical prices) and even place trades or manage strategies, programmatically. Enforce strong authentication (API keys or OAuth) and rate limiting on these APIs to ensure security and stability.
- **Week 69:** Implement an automated **audit & compliance self-check** system. Every night, run a batch job that verifies data integrity (e.g. ensure logs haven’t been tampered with, all trades are accounted for in P&L, etc.), checks that all scheduled tasks (like backups, report generation) completed successfully, and flags any discrepancies. If issues are found, trigger alerts to admins.
- **Week 70:** Conduct performance **scalability testing** on the platform. Simulate a high number of concurrent users, large volumes of data (e.g. many simultaneous trades or strategy runs), and long runtime durations to profile resource usage. Identify bottlenecks (CPU, memory, database, network) and optimize heavy operations like database queries, data serialization, or algorithmic loops. The

outcome should be a list of improvements (indexes, code optimizations, caching strategies) implemented to ensure the system can scale to enterprise loads.

- **Week 71:** Create an **advanced scenario analysis** module for risk management. Support running large-scale Monte Carlo simulations or historical replay simulations on portfolios to estimate risk distributions and potential outcomes. Integrate this with the risk engine so that results (like VaR from Monte Carlo) can be viewed alongside deterministic scenario results. This will likely require offloading heavy computation to background jobs or using efficient numerical libraries.
- **Week 72:** Add support for **financial derivatives** (if not already) focusing on futures and options. This involves pricing models for these instruments, handling their specific data (expiry dates, strike prices), and integrating their risk (Greeks, margin) into the risk engine. Extend the UI to allow viewing and trading of futures/options, and update the risk dashboards to include their exposures.
- **Week 73:** Develop an **AI-driven strategy generation** feature (initially as a deterministic demo). Allow the system to suggest or even create simple trading strategies based on historical data patterns or user goals. For example, using a rules-based AI or deterministic ML model to generate a strategy that the user can review. (At this stage, keep it deterministic and transparent—e.g. a decision tree based on past successful trades—so it's explainable.) Integrate this with the existing strategy backtester so users can immediately test the AI-generated strategies.
- **Week 74:** Implement **multi-user analytics** dashboards. If the platform is used by an organization with multiple users (traders, analysts), provide dashboards that aggregate performance and risk across all users or accounts, with the ability to filter by user. For example, an admin could see total firm P&L broken down by user, or risk exposures per user. Ensure permissions so that regular users can't see others' data unless allowed (linking back to RBAC from Year 1).
- **Week 75:** Enhance platform **observability** for operations. Integrate internal metrics collection (CPU, memory, queue lengths, etc.) and create admin dashboards for system health. Incorporate logs and metrics into a cohesive monitoring dashboard (perhaps using an embedded open-source monitoring UI, or exposing an endpoint for Prometheus/Grafana if used internally). This helps maintain reliability as the system scales.
- **Week 76:** Build an **onboarding wizard** for new users. This guided tour will introduce key features of UI2, help set up an initial watchlist or strategy, and provide a demo scenario (possibly using the deterministic Demo mode) to let users see how the system works. This will improve user adoption and ensure that new users don't feel lost given the platform's growing complexity.
- **Week 77:** Add **scenario-based Autopilot backtesting** capability. This means allowing Autopilot's decision-making algorithms to be run on historical data, similar to how strategies are backtested. The goal is to validate how the Autopilot would have performed under historical conditions or specific scenarios. This requires capturing Autopilot logic in a form that can be replayed deterministically on past market data.
- **Week 78:** Begin preparations for **open-sourcing** parts of the platform (if that is a goal). Identify which components or repositories can be open-sourced. This week, focus on code cleanup: remove or abstract away any sensitive information (e.g. API keys, secrets), ensure documentation is sufficient for external contributors, and establish contribution guidelines and a code of conduct. The aim is to position the project for potential community involvement in the next phase.

Year 2 – Q3: AI Workflows as Product & Custom Agents (Weeks 79–91)

- **Week 79:** Finalize the **Version 2.0 release candidate**. By this week, aim to have all major features planned for V2 implemented. Freeze any new feature work and concentrate on final QA testing and

addressing any outstanding issues or feedback from internal testing. Conduct specialized tests like **penetration testing** for security, and resolve any critical vulnerabilities or flaws.

- **Week 80:** Focus on **hardening and documentation**. Complete all end-user and developer documentation: finalize user guides, API references, and technical docs for new features. Add more end-to-end example use-cases in documentation (e.g. a sample workflow from creating a strategy to analyzing results) to demonstrate platform capabilities.
- **Week 81:** Launch a **beta release** of Version 2.0 to a group of selected users or an internal team. During this beta, closely monitor system logs and telemetry for any anomalies or performance issues not caught in testing. Collect user feedback on usability, bugs, and any missing features.
- **Week 82:** Implement improvements based on beta feedback. This could include adjusting certain UI flows that users found confusing, fixing bugs discovered during broader usage, and fine-tuning performance (e.g. if some pages were slow under real user data). Aim to address all high-priority feedback items this week.
- **Week 83:** Add **role-specific dashboards** to cater to different types of users (e.g. a Trader dashboard vs. a Risk Manager dashboard vs. an Admin dashboard). Each role's dashboard will highlight the information most relevant to them (traders see P&L and positions, risk managers see firm risk metrics, etc.). This makes the platform more immediately useful for each persona.
- **Week 84:** Develop an AI model **evaluation framework**. As more AI-driven features (like strategy generation or autopilot enhancements) are introduced, create a system to evaluate these models/strategies on historical data and measure performance metrics (win rate, Sharpe ratio, drawdown, etc.) in a controlled, offline manner. Store these evaluation results for comparison (so one can compare the performance of different strategy versions or AI models).
- **Week 85:** Strengthen integration tests for complex features. For example, create a comprehensive integration test suite for the multi-provider data aggregator (introduced in Year 1) to ensure it handles provider failures gracefully (simulate one provider going down and verify the system seamlessly fails over to another). Generally, use this week to bolster test coverage on any critical subsystems that have had many changes (Autopilot, risk engine, etc.), ensuring reliability.
- **Week 86:** Implement user **metadata preferences** functionality. Allow users to set and save personalized preferences such as theme (light/dark mode), layout density (compact vs. comfortable spacing), and default metrics or views. Store these preferences server-side (so they persist across devices). Ensure that applying preferences is done in a deterministic way (for instance, theme switching should not cause any unpredictable behavior in running strategies or data displays).
- **Week 87:** Prepare for a **cross-platform release** if a mobile/web version is planned. Finalize packaging for a mobile app or PWA (Progressive Web App) if applicable. Perform thorough testing across devices (desktop, tablet, mobile) to ensure that the experience is smooth and consistent. Update documentation and support materials to include any platform-specific notes (like how to install or use the mobile app).
- **Week 88:** Compile a compelling **investor/demo pitch** (if external funding or internal buy-in is needed for the next phase). Gather key metrics from the past development (performance improvements achieved, number of features, etc.), highlight user traction or community interest, and showcase product maturity. Create a pitch deck and demo script that emphasizes the platform's competitive strengths and readiness for growth.
- **Week 89:** Complete a thorough **multi-regulator compliance review**. Ensure that the platform meets the requirements of relevant regulatory bodies (for finance, this could be SEC, FINRA, etc., depending on region and domain). This might involve adding any missing features like data retention policies, specific reports, or user data privacy controls. Implement any final compliance-related changes identified by this review (for example, if a new rule requires capturing a certain type of trading data or providing an audit trail in a specific format).

- **Week 90:** Officially **release Version 2.0 (Stable)** to all users and/or open source the repository as planned. This is the culmination of the 2-year development efforts. Tag the repository for v2.0, publish release notes highlighting all the major features and improvements. Begin actively onboarding new users and, if open-sourced, welcome community contributors.
- **Week 91:** Provide **post-release support** and stabilization. Monitor for any incidents or bugs that escaped into the wild after release. Patch urgent issues promptly (hotfixes). Ensure the operations side is maintaining stability as user load potentially increases. Also, use this time to observe user behavior in production to gather ideas for further improvements.

Year 2 – Q4: Polish, Growth & Competition Readiness (Weeks 92–104)

- **Week 92:** Build out a **Plugin Marketplace** concept. Now that plugins exist (from Year 1) and possibly third parties can create them, develop a section of the platform where vetted third-party plugins can be listed and shared. Implement an *upload/registration process* for plugins with an admin review step before they become available to all users. This will foster a community and ecosystem around the platform.
- **Week 93:** Develop an advanced AI **Autopilot simulation mode**. Incorporate more sophisticated AI (potentially generative models or reinforcement learning agents) into Autopilot's decision logic, but run them in a sandboxed simulation mode first. The goal is to see if these AI-driven strategies can outperform the deterministic rules. Evaluate their performance against a deterministic baseline to ensure any adoption of AI is justified by clear improvements.
- **Week 94:** Add a **continuous backtesting infrastructure** that automatically runs in the background. For example, each night, run backtests for any new strategies created or any new parameters introduced (nightly regression tests for strategies). This helps catch any issues early and provides users with fresh performance stats each day. Ensure this doesn't overload the system – use scheduling and queue management to spread out load.
- **Week 95:** Optimize **subscription & license management** features, in case the platform will be offered commercially. Integrate a payment gateway and license key system to manage paid subscriptions or enterprise licenses. Enforce feature gating via RBAC or feature flags based on subscription level (e.g. certain advanced analytics might only be in a "Pro" tier). Make sure this is done in a secure and user-friendly way (clear upgrade paths, trials, etc.).
- **Week 96:** Implement dynamic **risk parameter optimization** tools. For users or algorithms that want to fine-tune risk settings, allow the system to suggest optimal risk parameter values based on historical performance and user-defined objectives (e.g. maximize return for a target max drawdown). This can leverage the optimization engine from Week 54, now applied specifically to risk settings.
- **Week 97:** Enhance **cross-asset margin & collateral management**. If users trade multiple asset classes, allow collateral in one class to offset margin requirements in another where appropriate (cross-margining). For example, holdings in a stock portfolio could offset some margin requirements for futures positions. Implement these rules and test cross-product exposure calculations to ensure the risk engine correctly accounts for offsets and doesn't double-count exposure.
- **Week 98:** Add support for **decentralized finance (DeFi) connectors** (if relevant to the domain). Integrate data feeds or trading connectivity to decentralized exchanges (DEXes) or blockchain-based assets. This could include reading on-chain data, placing orders via smart contract interactions, etc. Unify this with the existing data aggregator and trading interfaces so that, to the user, trading a decentralized asset feels similar to trading a centralized exchange asset.

- **Week 99:** Undertake a final round of performance optimization and code **refactoring**. After two years of continuous development, revisit parts of the codebase to improve maintainability: eliminate any remaining hacks or tech debt, refactor modules for clarity, and improve testability. Also, prepare for an external **security audit** or code audit by ensuring documentation and code comments are up-to-date and the codebase is easy to navigate for auditors.
 - **Week 100:** Prepare the project for **long-term maintenance**. This includes updating all dependencies to their latest stable versions (to start Year 3 without outdated libraries), modularizing code where necessary to make future updates easier, and writing thorough developer documentation/comments for complex parts of the system. Essentially ensure that if new team members or open-source contributors come in after this point, they can understand and work with the system efficiently.
 - **Week 101:** Begin research and exploration of **next-generation features** that could give the platform a competitive edge beyond this roadmap. This might involve investigating machine-learning-driven trading strategies, reinforcement learning-based optimization, or other cutting-edge approaches. Prototype integration with one or two such technologies (in an isolated branch or environment) to assess feasibility. These are exploratory “R&D” tasks to seed the roadmap for Year 3.
 - **Week 102:** Assess and incorporate community contributions (if open-sourced). By now, external users or contributors may have submitted pull requests or issues. Set up a process for reviewing PRs, run community testing for those contributions, and merge those that meet quality standards. Address any outstanding issues reported by the community. This ensures the project remains healthy and grows with help from outside.
 - **Week 103:** Formulate a **Year 3 roadmap** based on everything learned so far. Gather user feedback, analytics on feature usage, and the outcomes of Week 101 R&D to decide on the next major priorities. This plan might include scaling to more users, introducing new asset classes, deeper AI integration, or other innovations. Summarize the potential features and improvements for the future, based on identified gaps or new opportunities.
 - **Week 104:** Finalize a **two-year project report** and hold a retrospective. Compile a summary of achievements over the 2-year period (features delivered, performance gains, user growth, etc.). Reflect on what went well and what could be improved in the development process. Share this report with stakeholders and the community. Use insights from this reflection to refine processes and goals as the team moves beyond the initial 2-year plan.
-